

ui.video 全面详细阐述

`ui.video` 是 NiceGUI 框架中用于**视频播放**的核心组件，基于 HTML5 `<video>` 元素封装，继承了原生视频播放的全部核心能力，同时融合了 NiceGUI 组件的响应式特性、样式定制能力及事件绑定机制。该组件支持主流视频格式、自定义控制界面、播放状态监听及程式化控制，适用于视频展示、在线课程、媒体播放器、视频标注等场景。以下从核心特性、使用场景、配置选项、事件系统、进阶用法等方面展开详细说明。

一、核心概述

1. 功能定位

- 基础能力：加载并播放本地 / 网络视频文件，支持暂停、音量调节、进度控制、全屏播放等核心功能；
- 格式支持：兼容 HTML5 `<video>` 标准格式（MP4、WebM、OGG 等）；
- 样式定制：支持原生控件样式、隐藏原生控件并通过 NiceGUI 组件自定义播放面板；
- 状态监听：支持播放、暂停、结束、加载完成、进度更新等完整事件；
- 编程控制：支持通过代码触发播放、暂停、跳转进度、调节音量、切换视频源等操作；
- 扩展能力：支持嵌套叠加层（如字幕、标注、交互按钮），兼容 Lottie 动画、SVG 等增强元素。

2. 核心优势

优势点	说明
轻量化集成	基于原生 HTML5 视频 API，无需额外依赖（如 Flash、第三方播放器库）；
多源支持	支持网络 URL、本地文件、Base64 编码等多种视频源；
样式灵活	可使用原生控件，也可自定义播放面板（与 NiceGUI 组件无缝联动）；
事件与状态完备	提供完整的播放状态回调，支持精准控制与状态同步；
响应式适配	支持 Tailwind/Quasar 样式类，可适配不同屏幕尺寸（桌面端 / 移动端）；
叠加层兼容	支持嵌套 <code>ui.label</code> 、 <code>ui.html</code> 、 <code>ui.button</code> 等组件，实现字幕、标注等增强功能；
生态兼容	可与 <code>ui.interactive_image</code> 、 <code>ui.slider</code> 等组件联动，构建复杂媒体交互场景。

3. 支持的视频格式

格式	扩展名	浏览器兼容性	适用场景
MP4	<code>.mp4</code>	所有现代浏览器 (Chrome/Firefox/Edge/Safari)	最常用格式，推荐优先使用
WebM	<code>.webm</code>	Chrome/Firefox/Edge，部分支持 Safari	开源格式，压缩比高，适合网页视频
OGG	<code>.ogg</code> / <code>.ogv</code>	Chrome/Firefox/Edge，不支持 Safari	开源格式，适合小型视频片段
MOV	<code>.mov</code>	Safari 原生支持，Chrome/Firefox 需额外解码	苹果生态常用格式

格式	扩展名	浏览器兼容性	适用场景
AV1	.av1	现代浏览器逐步支持（Chrome 90+/Firefox 85+）	高效压缩格式，适合 4K/8K 视频

注：MP4 格式（H.264 编码）兼容性最广，建议作为首选格式；WebM 格式压缩效率更高，可作为备选（需考虑 Safari 兼容性）。

二、基础使用与视频源支持

`ui.video` 的核心参数为 `source`，支持多种视频源输入，用法与 `ui.audio` 高度一致，降低学习成本。以下是不同视频源的使用示例：

1. 网络视频（URL 源）

直接通过网络 URL 加载视频文件，适用于公开视频资源（如 CDN 上的视频、在线视频平台的嵌入链接）。

```
from nicegui import ui

# 加载网络 MP4 视频（NiceGUI 官方测试视频）
ui.video('https://nicegui.io/examples/video.mp4')

ui.run()
```

2. 本地视频文件

通过本地文件路径加载视频，需注意文件路径的正确性（推荐使用相对路径）。

```
from nicegui import ui

# 加载本地视频文件（相对路径，以入口文件 main.py 为基准）
ui.video('static/videos/demo.mp4')

# 加载绝对路径视频（需确保运行环境有权限访问）
# ui.video('/home/user/videos/presentation.mp4')

ui.run()
```

- 路径说明：
 - 相对路径：推荐将视频文件放在 `static` 文件夹下（NiceGUI 默认静态资源目录），直接通过 `static/[文件夹]/[文件名]` 访问；
 - 绝对路径：适用于固定路径的视频文件，但跨环境兼容性较差，不推荐优先使用。

3. Base64 编码视频

将小型视频文件编码为 Base64 字符串直接嵌入，无需额外文件依赖，适用于短片段视频（如产品演示、小动画）。

```
from nicegui import ui
import base64
```

```
# 读取本地视频文件并编码为 Base64
def video_to_base64(file_path):
    with open(file_path, 'rb') as f:
        base64_data = base64.b64encode(f.read()).decode('utf-8')
    # 前缀格式: data:video/[格式];base64,
    return f'data:video/mp4;base64,{base64_data}'

# 转换本地 MP4 为 Base64 并加载（建议仅用于 <10MB 的小型视频）
base64_video = video_to_base64('static/videos/short_demo.mp4')
ui.video(base64_video)

ui.run()
```

- **注意：**Base64 编码会使文件体积增加约 33%，且加载时需一次性解码，仅适用于小型视频（建议 <10MB）；大型视频优先使用 URL 或本地文件路径。

4. 响应式视频源（动态切换）

结合 NiceGUI 的响应式机制，实现视频源的动态切换（如通过按钮切换不同视频片段）。

```
from nicegui import ui

# 响应式变量存储视频源
video_source = ui.reactive('https://nicegui.io/examples/video.mp4')

# 视频组件绑定响应式源
video = ui.video().bind_source(video_source)

# 按钮切换视频源
ui.row([
    ui.button('播放视频1', on_click=lambda: setattr(video_source, 'value',
'https://nicegui.io/examples/video.mp4')),
    ui.button('播放视频2', on_click=lambda: setattr(video_source, 'value',
'https://example.com/another_video.mp4')),
])

ui.run()
```

三、核心配置选项

`ui.video` 提供丰富的配置属性，用于控制视频播放行为、控件样式、响应式状态等，核心属性如下（继承自 `ui.element`，并新增视频专属配置）：

属性名	类型	说明	默认值	
<code>source</code>	<code>BindableProperty</code>	视频源（支持 URL、本地路径、Base64 字符串），可绑定响应式变量；	<code>None</code>	
<code>autoplay</code>	<code>bool</code>	视频加载完成后是否自动播放（浏览器限制：需用户交互后生效）；	<code>False</code>	

属性名	类型	说明	默认值	
loop	bool	是否循环播放视频；	False	
muted	bool	是否默认静音；	False	
controls	bool	是否显示原生播放控件（播放 / 暂停、进度条、音量、全屏等）；	True	
preload	str	预加载策略：'none'（不预加载）、'metadata'（仅预加载元数据）、'auto'（自动预加载）；	'auto'	
volume	float	初始音量（0.0~1.0，0 为静音，1 为最大音量）；	1.0	
poster	`str`	None`	视频封面图（未播放时显示的图像），支持 URL、本地路径、Base64；	None
playsinline	bool	移动端是否允许 inline 播放（不自动全屏）；	True	
fullscreen_options	`dict`	None`	全屏播放选项（如 {'navigationUI': 'hide'} 隐藏导航栏）；	None
classes	Classes[Self]	应用于视频元素的 CSS 类（支持 Tailwind、Quasar 类）；	''	
style	Style[Self]	内联 CSS 样式（如 width: 100%; height: auto;）；	''	
html_id	str	HTML DOM 元素 ID（v2.16.0+ 支持）；	''	
visible	BindableProperty	组件是否可见（可绑定响应式变量）；	True	

配置示例（自定义播放行为与样式）

```
from nicegui import ui

# 配置：封面图、自动播放（需用户交互）、循环、默认静音、隐藏原生控件、自适应尺寸
ui.video(
    source='https://nicegui.io/examples/video.mp4',
    poster='https://picsum.photos/id/237/800/450', # 封面图（未播放时显示）
    autoplay=True,
    loop=True,
    muted=True,
    controls=False,
    preload='metadata', # 仅预加载时长、尺寸等元数据
    volume=0.8,
    playsinline=True, # 移动端 inline 播放
    fullscreen_options={'navigationUI': 'hide'}, # 全屏时隐藏导航栏
).classes('w-full max-w-4xl mx-auto border-4 border-gray-200 rounded-lg shadow-lg')

# 添加自定义播放按钮（用户交互后触发自动播放）
ui.button('开始播放', on_click=lambda: ui.query('video').run_method('play()')).classes('mt-4 mx-auto block')
```

```
ui.run()
```

四、核心事件系统

`ui.video` 支持完整的视频播放状态事件，可实时监听播放、暂停、结束、加载、进度更新等状态变化，事件回调中可获取当前播放进度、音量、视频尺寸等关键信息。

1. 支持的事件类型与参数

事件名	触发时机	事件参数 (e) 关键属性
<code>play</code>	视频开始播放时触发（首次播放、暂停后恢复）	<code>current_time</code> ：当前播放时间（秒）； <code>duration</code> ：视频总时长（秒）； <code>volume</code> ：当前音量； <code>video_width / video_height</code> ：视频原始尺寸
<code>pause</code>	视频暂停时触发	同 <code>play</code>
<code>ended</code>	视频播放结束时触发（非循环模式下）	同 <code>play</code>
<code>loadeddata</code>	视频第一帧数据加载完成（可播放）时触发	<code>duration</code> ：视频总时长； <code>video_width / video_height</code> ：视频尺寸； <code>poster</code> ：封面图 URL；
<code>timeupdate</code>	播放进度更新时触发（约每秒 4 次）	<code>current_time</code> ：当前播放时间； <code>progress</code> ：播放进度（0.0~1.0）；
<code>volumechange</code>	音量变化或静音切换时触发	<code>volume</code> ：当前音量（0.0~1.0）； <code>muted</code> ：是否静音；
<code>resize</code>	视频尺寸变化时触发（如全屏 / 退出全屏）	<code>video_width / video_height</code> ：新的视频尺寸； <code>fullscreen</code> ：是否全屏；
<code>fullscreenchange</code>	全屏状态变化时触发	<code>fullscreen</code> ： <code>True</code> （进入全屏）/ <code>False</code> （退出全屏）；
<code>error</code>	视频加载或播放失败时触发	<code>error</code> ：错误信息对象（错误代码、描述）；
<code>canplay</code>	视频缓冲足够，可开始播放时触发	同 <code>loadeddata</code>
<code>waiting</code>	视频缓冲中（暂时无法播放）时触发	无额外参数

事件名	触发时机	事件参数 (e) 关键属性
<code>progress</code>	视频缓冲进度更新时触发	<code>buffered</code> : 已缓冲的时间范围数组 (如 <code>[[0, 10], [20, 30]]</code> 表示 0-10 秒、20-30 秒已缓冲) ;
<code>loadedmetadata</code>	视频元数据 (时长、尺寸) 加载完成时触发	同 <code>loadeddata</code>

2. 事件绑定示例（核心场景）

示例 1：监听播放状态与进度

```
from nicegui import ui

video = ui.video('https://nicegui.io/examples/video.mp4').classes('w-full max-w-4xl mx-auto')

# 状态显示面板
with ui.card().classes('w-full max-w-4xl mx-auto mt-4 p-4'):
    status_label = ui.label('状态: 未播放').classes('font-bold')
    progress_label = ui.label('进度: 0%')
    info_label = ui.label('视频信息: 加载中...')

# 监听播放事件
@video.on('play')
def on_play(e):
    status_label.set_text(f'状态: 播放中')
    info_label.set_text(f'视频信息: {e.video_width}x{e.video_height}px | 总时长: {e.duration:.1f}秒')

# 监听暂停事件
@video.on('pause')
def on_pause(e):
    status_label.set_text(f'状态: 已暂停 | 当前进度: {e.current_time:.1f}秒')

# 监听播放结束事件
@video.on('ended')
def on_ended(e):
    status_label.set_text('状态: 播放结束')

# 监听进度更新事件
@video.on('timeupdate')
def on_timeupdate(e):
    progress = (e.current_time / e.duration) * 100 if e.duration else 0
    progress_label.set_text(f'进度: {progress:.1f}%')

# 监听全屏状态变化
@video.on('fullscreenchange')
def on_fullscreen_change(e):
    status_label.set_text(f'状态: {"全屏播放中" if e.fullscreen else "播放中"}')
```

```
# 监听加载失败事件
@video.on('error')
def on_error(e):
    status_label.set_text(f'状态: 加载失败 | 错误: {e.error}')

ui.run()
```

示例 2：视频缓冲进度监听

```
from nicegui import ui

video = ui.video('https://nicegui.io/examples/video.mp4').classes('w-full')

# 缓冲进度显示
buffer_label = ui.label('缓冲进度: 0%').classes('mt-2')

@video.on('progress')
def on_progress(e):
    if not e.buffered:
        return
    # 获取已缓冲的最大时间（最后一段缓冲的结束时间）
    buffered_end = e.buffered[-1][1]
    buffer_progress = (buffered_end / e.duration) * 100 if e.duration else 0
    buffer_label.set_text(f'缓冲进度: {buffer_progress:.1f}% (已缓冲至 {buffered_end:.1f} 秒)')

ui.run()
```

五、编程式控制方法

`ui.video` 提供一系列方法用于通过代码控制视频播放行为，支持播放、暂停、跳转进度、调节音量、切换视频源等操作，核心方法如下：

方法名	作用	参数说明
<code>play()</code>	开始播放视频（用户交互后生效）	无参数；返回 <code>None</code> （异步执行）
<code>pause()</code>	暂停当前播放	无参数；返回 <code>None</code>
<code>stop()</code>	停止播放并重置进度到起始位置（0 秒）	无参数；返回 <code>None</code>
<code>seek(seconds)</code>	跳转播放进度到指定时间点	<code>seconds</code> ：目标时间（秒）；超出时长则跳转到结束位置；返回 <code>None</code>
<code>set_volume(volume)</code>	设置音量（0.0~1.0）	<code>volume</code> ：目标音量（浮点数，超出范围自动截断）；返回 <code>None</code>
<code>set_muted(muted)</code>	设置是否静音	<code>muted</code> ： <code>True</code> （静音）/ <code>False</code> （取消静音）；返回 <code>None</code>

方法名	作用	参数说明
<code>set_source(source)</code>	切换视频源	<code>source</code> ：新视频源（URL / 本地路径 / Base64）；返回 <code>None</code>
<code>enter_fullscreen()</code>	进入全屏播放	无参数；返回 <code>None</code>
<code>exit_fullscreen()</code>	退出全屏播放	无参数；返回 <code>None</code>
<code>toggle_fullscreen()</code>	切换全屏 / 非全屏状态	无参数；返回 <code>None</code>
<code>get_current_time()</code>	获取当前播放时间（秒）	无参数；返回 <code>float</code> 类型
<code>get_duration()</code>	获取视频总时长（秒）（需加载完成后调用）	无参数；返回 <code>float</code> 类型
<code>get_volume()</code>	获取当前音量（0.0~1.0）	无参数；返回 <code>float</code> 类型
<code>is_playing()</code>	判断视频是否正在播放	无参数；返回 <code>bool</code> 类型
<code>is_muted()</code>	判断是否静音	无参数；返回 <code>bool</code> 类型
<code>is_fullscreen()</code>	判断是否处于全屏状态	无参数；返回 <code>bool</code> 类型
<code>force_reload()</code>	强制重新加载视频源	无参数；返回 <code>None</code>

方法使用示例（自定义视频播放器）

结合 NiceGUI 组件（按钮、滑块、进度条）实现功能完整的自定义视频播放器，替代原生控件：

```
from nicegui import ui

class CustomVideoPlayer:
    def __init__(self, video_source, poster=None):
        self.source = video_source
        self.poster = poster
        self.video = None
        self.duration = 0 # 视频总时长（秒）
        self._build_ui()

    def _build_ui(self):
        # 视频组件（隐藏原生控件）
        self.video = ui.video(
            source=self.source,
            poster=self.poster,
            controls=False,
            playsinline=True,
        ).classes('w-full max-w-4xl mx-auto bg-black rounded-lg')

        # 自定义控制面板（半透明悬浮在视频下方）
        with ui.row().classes('absolute bottom-0 left-0 right-0 bg-black/60 text-white p-3 gap-2 items-center'):
            # 播放/暂停按钮
```



```

        self.play_btn = ui.button('▶', on_click=self.toggle_play).classes('w-10 h-10
rounded-full bg-blue-500 text-white')

        # 进度条
        self.progress_bar = ui.slider(min=0, max=100, value=0, step=0.1).classes('flex-
1')

        self.progress_bar.on('change', self.seek_to_progress)

        # 时间显示
        self.time_label = ui.label('00:00 / 00:00').classes('w-24 text-center')

        # 音量控制
        self.volume_btn = ui.button('🔊', on_click=self.toggle_mute).classes('w-10 h-10
rounded-full')
        self.volume_slider = ui.slider(min=0, max=1, value=0.7, step=0.01).classes('w-
24')

        self.volume_slider.on('change', lambda e: self.video.set_volume(e.value))

        # 全屏按钮
        ui.button('⏏', on_click=self.toggle_fullscreen).classes('w-10 h-10 rounded-
full')

        # 绑定视频事件
        self._bind_events()

    def _bind_events(self):
        # 加载完成：获取总时长和尺寸
        @self.video.on('loadeddata')
        def on_loaded(e):
            self.duration = e.duration
            self.progress_bar.max = self.duration
            self.time_label.set_text(self._format_time(0) + ' / ' +
self._format_time(self.duration))

        # 进度更新：同步进度条和时间显示
        @self.video.on('timeupdate')
        def on_timeupdate(e):
            current_time = e.current_time
            self.progress_bar.set_value(current_time)
            self.time_label.set_text(self._format_time(current_time) + ' / ' +
self._format_time(self.duration))

        # 播放状态更新：切换按钮图标
        @self.video.on('play')
        def on_play(e):
            self.play_btn.set_text('⏏')

        @self.video.on('pause')
        def on_pause(e):
            self.play_btn.set_text('▶')

        # 音量变化：同步音量滑块和按钮
        @self.video.on('volumechange')

```

```

def on_volume_change(e):
    self.volume_slider.set_value(e.volume)
    self.volume_btn.set_text('🔊' if e.muted else '🔊')

# 全屏状态更新: 切换全屏按钮图标
@self.video.on('fullscreenchange')
def on_fullscreen_change(e):
    fullscreen_btn = ui.query('.q-btn').last()
    fullscreen_btn.set_text('❌' if e.fullscreen else '🖥️')

def toggle_play(self):
    # 切换播放/暂停
    if self.video.is_playing():
        self.video.pause()
    else:
        self.video.play()

def toggle_mute(self):
    # 切换静音
    self.video.set_muted(not self.video.is_muted())

def seek_to_progress(self, e):
    # 进度条跳转
    self.video.seek(e.value)

def toggle_fullscreen(self):
    # 切换全屏
    if self.video.is_fullscreen():
        self.video.exit_fullscreen()
    else:
        self.video.enter_fullscreen()

@staticmethod
def _format_time(seconds):
    # 格式化时间: 秒 → 分:秒 (如 125 秒 → 02:05, 超过1小时则 时:分:秒)
    hours = int(seconds // 3600)
    minutes = int((seconds % 3600) // 60)
    secs = int(seconds % 60)
    if hours > 0:
        return f'{hours:02d}:{minutes:02d}:{secs:02d}'
    return f'{minutes:02d}:{secs:02d}'

# 初始化自定义播放器 (带封面图)
player = CustomVideoPlayer(
    video_source='https://nicegui.io/examples/video.mp4',
    poster='https://picsum.photos/id/103/1280/720'
)

ui.run()

```

六、进阶用法（核心场景落地）

场景 1：视频标注工具（结合叠加层与交互）

基于 `ui.video` 的叠加层嵌套能力，实现视频播放过程中的实时标注（如添加文字、图形标注）：

```
from nicegui import ui

class VideoAnnotator:
    def __init__(self, video_source):
        self.video_source = video_source
        self.video = None
        self.annotations = [] # 存储标注: [{time, text, position}]
        self._build_ui()

    def _build_ui(self):
        # 视频容器（相对定位，用于叠加层）
        with ui.container().classes('relative w-full max-w-4xl mx-auto'):
            # 视频组件
            self.video = ui.video(self.video_source).classes('w-full')

            # 标注输入面板（悬浮在视频右上角）
            with ui.card().classes('absolute top-2 right-2 bg-white/90 p-2 rounded-md z-10'):
                ui.label('视频标注').classes('font-bold')
                self.annotation_input = ui.input(placeholder='输入标注内容').classes('w-48')
                ui.button('添加标注', on_click=self.add_annotation).classes('w-full')

            # 标注列表
            ui.label('标注列表').classes('mt-4 font-bold text-xl')
            self.annotations_list = ui.list().classes('w-full max-w-4xl mx-auto')

            # 绑定视频事件（进度更新时高亮当前标注）
            @self.video.on('timeupdate')
            def highlight_current_annotation(e):
                current_time = e.current_time
                # 高亮当前时间点前后1秒内的标注
                for item in self.annotations_list.children:
                    annotation_time = item.metadata['time']
                    if abs(current_time - annotation_time) < 1:
                        item.classes('bg-yellow-100')
                    else:
                        item.classes('bg-transparent')

    def add_annotation(self):
        text = self.annotation_input.value.strip()
        if not text:
            ui.notify('标注内容不能为空!', type='error')
            return

        current_time = self.video.get_current_time()
        # 记录标注（当前时间、内容、位置）
```

```

        annotation = {
            'time': current_time,
            'text': text,
            'position': 'top-right' # 可扩展为自定义位置
        }
        self.annotations.append(annotation)

        # 添加到标注列表（点击可跳转到对应时间点）
        list_item = self.annotations_list.add_item(
            f'[{self._format_time(current_time)}] {text}',
            on_click=lambda t=current_time: self.video.seek(t)
        )
        list_item.metadata['time'] = current_time

        # 在视频上添加临时标注（显示5秒）
        with self.video:
            label = ui.label(text).classes(
                'absolute top-4 right-4 bg-red-500 text-white px-2 py-1 rounded-md text-sm z-10'
            )
            ui.timer(5, label.delete) # 5秒后自动删除

        self.annotation_input.clear()
        ui.notify('标注添加成功! ')

    @staticmethod
    def _format_time(seconds):
        minutes = int(seconds // 60)
        secs = int(seconds % 60)
        return f'{minutes:02d}:{secs:02d}'

# 初始化视频标注工具
annotator = VideoAnnotator('https://nicegui.io/examples/video.mp4')

ui.run()

```

场景 2：视频列表与播放记忆

实现多视频列表切换，并记录每个视频的播放进度（下次播放时自动跳转到上次位置）：

```

from nicegui import ui
import json
import os

# 存储播放进度的文件
PROGRESS_FILE = 'video_progress.json'

class VideoPlayerWithHistory:
    def __init__(self, video_list):
        self.video_list = video_list # 格式: [{id, title, source}, ...]
        self.current_video_id = None
        self.playback_progress = self._load_progress() # 加载历史进度

```

```

self.video = None
self._build_ui()

def _load_progress(self):
    # 从文件加载播放进度
    if os.path.exists(PROGRESS_FILE):
        with open(PROGRESS_FILE, 'r') as f:
            return json.load(f)
    return {}

def _save_progress(self, video_id, progress):
    # 保存播放进度到文件
    self.playback_progress[video_id] = progress
    with open(PROGRESS_FILE, 'w') as f:
        json.dump(self.playback_progress, f)

def _build_ui(self):
    # 左右布局: 视频列表 + 播放器
    with ui.row().classes('w-full max-w-6xl mx-auto'):
        # 视频列表 (左侧)
        with ui.column().classes('w-64 pr-4 overflow-y-auto max-h-[600px]'):
            ui.label('视频列表').classes('font-bold text-xl mb-2')
            for video in self.video_list:
                ui.button(
                    video['title'],
                    on_click=lambda v=video: self.load_video(v),
                    classes='w-full text-left mb-1 p-2 rounded hover:bg-blue-100'
                ).bind_classes({'bg-blue-500 text-white': self.current_video_id ==
video['id']})

        # 播放器 (右侧)
        with ui.column().classes('flex-1'):
            self.video = ui.video().classes('w-full border-2 border-gray-200 rounded-
1g')

            self.video.on('timeupdate', self._on_progress_update)
            self.video.on('ended', lambda: self._save_progress(self.current_video_id,
0)) # 播放结束后重置进度

def load_video(self, video):
    # 加载选中的视频
    self.current_video_id = video['id']
    self.video.set_source(video['source'])

    # 跳转到上次播放进度 (如果存在)
    progress = self.playback_progress.get(video['id'], 0)
    if progress > 0:
        ui.notify(f'恢复上次播放进度: {self._format_time(progress)}')
        self.video.seek(progress)

    # 自动播放
    self.video.play()

def _on_progress_update(self, e):

```

```

        # 实时保存播放进度（每3秒保存一次，避免频繁IO）
        if self.current_video_id is None:
            return
        current_time = e.current_time
        # 仅在进度变化且非最后1秒时保存
        if current_time < e.duration - 1:
            ui.timer(3, lambda: self._save_progress(self.current_video_id, current_time),
once=True)

    @staticmethod
    def _format_time(seconds):
        minutes = int(seconds // 60)
        secs = int(seconds % 60)
        return f'{minutes:02d}:{secs:02d}'

# 示例视频列表
video_list = [
    {'id': '1', 'title': '视频1: 产品介绍', 'source':
'https://nicegui.io/examples/video.mp4'},
    {'id': '2', 'title': '视频2: 操作教程', 'source': 'https://example.com/tutorial.mp4'},
    {'id': '3', 'title': '视频3: 常见问题', 'source': 'https://example.com/faq.mp4'},
]

# 初始化播放器
player = VideoPlayerWithHistory(video_list)

ui.run()

```

场景 3：结合 Lottie 动画的视频交互

在视频上添加 Lottie 动态动画作为交互元素（如播放按钮、加载动画、标注提示）：

```

from nicegui import ui

# 引入 Lottie 播放器脚本
ui.add_body_html('<script src="https://unpkg.com/@lottiefiles/lottie-
player@latest/dist/lottie-player.js"></script>')

# 视频容器（相对定位）
with ui.container().classes('relative w-full max-w-4xl mx-auto'):
    video = ui.video('https://nicegui.io/examples/video.mp4', controls=False).classes('w-
full rounded-lg')

    # 中心播放按钮（Lottie 动画）
    play_animation = ui.html(''
        <lottie-player
            src="https://assets1.lottiefiles.com/packages/lf20_9zqk0f.json"
            loop autoplay
            style="position: absolute; top: 50%; left: 50%; transform: translate(-50%,
-50%); width: 80px; height: 80px; cursor: pointer;"
        />
    '', sanitize=False)

```

```

# 加载动画（初始显示，视频加载完成后隐藏）
load_animation = ui.html('''
    <lottie-player
      src="https://assets1.lottiefiles.com/packages/lf20_5t3w2j.json"
      loop autoplay
      style="position: absolute; top: 50%; left: 50%; transform: translate(-50%,
-50%); width: 60px; height: 60px;"
    />
''', sanitize=False)

# 视频加载完成后隐藏加载动画
@video.on('loadeddata')
def on_loaded():
    load_animation.classes('hidden')

# 点击播放动画触发视频播放
play_animation.on('click', lambda: (video.play(), play_animation.classes('hidden')))

# 视频暂停时显示播放动画
@video.on('pause')
def on_pause():
    play_animation.classes('block')

# 视频播放时隐藏播放动画
@video.on('play')
def on_play():
    play_animation.classes('hidden')

ui.run()

```

七、注意事项与优化建议

1. 自动播放限制

- 现代浏览器（Chrome、Firefox、Safari 等）限制视频自动播放（`autoplay=True`）：需用户先进行交互（点击页面、输入等）后，自动播放才能生效；
- 解决方案：在页面添加明显的“开始播放”按钮，用户点击后通过 `video.play()` 触发播放（如自定义播放器场景）。

2. 视频加载与缓冲优化

- 大型视频文件建议使用流式传输协议（如 HLS、DASH），但 `ui.video` 原生不支持，需结合第三方库（如 `hls.js`）实现：

```

# 示例：结合 hls.js 支持 HLS 流式视频
from nicegui import ui

# 引入 hls.js 脚本
ui.add_head_html('<script
src="https://cdn.jsdelivr.net/npm/hls.js@1.4.14/dist/hls.min.js"></script>')

```

```

# 创建视频元素
video = ui.video().classes('w-full')
video_id = video.html_id

# 编写 JavaScript 初始化 HLS
ui.add_body_html(f'''
<script>
    if (Hls.isSupported()) {{
        const hls = new Hls();
        hls.loadSource('https://example.com/stream.m3u8'); // HLS 流地址
        hls.attachMedia(document.getElementById('{video_id}'));
    }} else if
(document.getElementById('{video_id}').canPlayType('application/vnd.apple.mpegurl'))
{{
    // Safari 原生支持 HLS
    document.getElementById('{video_id}').src = 'https://example.com/stream.m3u8';
}}
</script>
''')

ui.run()

```

- 预加载策略：大型视频使用 `preload='metadata'` 仅预加载元数据，减少初始加载时间；小型视频可使用 `preload='auto'` 提升用户体验。

3. 跨域问题

- 加载跨域视频文件时，需确保服务端设置 `Access-Control-Allow-Origin` 响应头，否则可能出现加载失败或无法获取视频元数据；
- 部署时，若视频文件与应用不同域名，需配置 CORS 规则（如 Nginx 中添加 `add_header Access-Control-Allow-Origin *`；）。

4. 性能优化

- 避免同时播放多个视频（导致 CPU / 内存占用过高，卡顿）；
- 不需要的视频组件及时调用 `delete()` 销毁，释放资源；
- 视频文件建议进行压缩优化（如降低分辨率、调整比特率），平衡画质与加载速度；
- 移动端优先使用 `playsinline=True`，避免自动全屏导致的交互体验下降。

5. 兼容性处理

- 老旧浏览器（如 IE）不支持 HTML5 `<video>`，需提供视频下载链接作为降级方案；
- 不同浏览器对视频格式的支持不同，建议同时提供 MP4 和 WebM 格式（通过 `<source>` 标签切换）：


```
# 多格式兼容示例
ui.html(`
<video controls class="w-full">
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  您的浏览器不支持视频播放，请下载视频：<a href="video.mp4">下载 MP4</a>
</video>
`, sanitize=False)
```

6. 移动端适配

- 移动端浏览器的原生控件样式可能与桌面端不一致，建议使用自定义控件保证跨端一致性；
- 移动端需注意视频播放时的锁屏控制（显示播放状态、进度条），可通过 HTML5 `MediaSession` API 实现（需额外编写 JavaScript）；
- 避免视频尺寸超出屏幕，使用 `classes('w-full height-auto')` 确保自适应。

7. 版权与安全

- 保护付费视频资源：避免直接暴露视频 URL（可使用临时签名 URL、防盗链技术）；
- 禁止视频下载：可通过隐藏下载按钮（原生控件不支持，需自定义控件）、使用加密视频格式等方式，但无法完全阻止（需结合服务端授权）。

总结

`ui.video` 是 NiceGUI 中功能完备、灵活性强的视频播放组件，基于 HTML5 原生视频 API 封装，支持多源视频加载、原生 / 自定义控件、播放状态监听及程式化控制。其核心优势在于与 NiceGUI 生态的无缝融合，可快速实现从简单视频展示到复杂交互场景（如自定义播放器、视频标注、播放记忆）的各类需求。

掌握该组件的关键在于：

1. 理解视频源的多种输入方式及适用场景（URL / 本地 / Base64）；
2. 熟练运用配置属性控制播放行为（自动播放、循环、封面图等）；
3. 结合事件系统与程式化方法，实现自定义交互逻辑；
4. 规避浏览器限制（自动播放、跨域）与兼容性问题；
5. 利用叠加层能力扩展视频的增强功能（标注、动态交互元素）。